In [1]:
```
%pip install -q tensorflow keras pandas scikit-learn gymnasium
```

```
[notice] A new release of pip is available: 25.1.1 -> 25.2
[notice] To update, run: pip install --upgrade pip
Note: you may need to restart the kernel to use updated packages.
```

Module 6 Assignment - PPO Experimentation

> Submitted by Phillip Amir Esguerra

In [2]:
```
import keras
import numpy as np
import tensorflow as tf
import gymnasium as gym
import scipy.signal
import time

from keras import layers
from gymnasium.core import Env
```

Setting Up Code

In [3]:
```
def mlp(x, sizes, activation=keras.activations.tanh, output_activation
    """Builds a simple feedforward MLP"""
    for size in sizes[:-1]:
        x = layers.Dense(size, activation=activation)(x)
    return layers.Dense(sizes[-1], activation=output_activation)(x)

def logprobabilities(logits, a, env: Env):
    # Compute the log-probabilities of taking actions a by using the l
    logprobabilities_all = keras.ops.log_softmax(logits)
    num_actions = env.action_space.n
    logprobability = keras.ops.sum(
        keras.ops.one_hot(a, num_actions) * logprobabilities_all, axis
    )
    return logprobability

def discounted_cumulative_sums(x, discount):
    # Discounted cumulative sums of vectors for computing rewards-to-g
    return scipy.signal.lfilter([1], [1, float(-discount)], x[::-1], a

class Buffer:
    # Buffer for storing trajectories
    def __init__(self, observation_dimensions, size, gamma=0.99, lam=0
```

```
        # Buffer initialization
        self.observation_buffer = np.zeros(
            (size, observation_dimensions), dtype=np.float32
        )
        self.action_buffer = np.zeros(size, dtype=np.int32)
        self.advantage_buffer = np.zeros(size, dtype=np.float32)
        self.reward_buffer = np.zeros(size, dtype=np.float32)
        self.return_buffer = np.zeros(size, dtype=np.float32)
        self.value_buffer = np.zeros(size, dtype=np.float32)
        self.logprobability_buffer = np.zeros(size, dtype=np.float32)
        self.gamma, self.lam = gamma, lam
        self.pointer, self.trajectory_start_index = 0, 0

    def store(self, observation, action, reward, value, logprobability
        # Append one step of agent-environment interaction
        self.observation_buffer[self.pointer] = observation
        self.action_buffer[self.pointer] = action
        self.reward_buffer[self.pointer] = reward
        self.value_buffer[self.pointer] = value
        self.logprobability_buffer[self.pointer] = logprobability
        self.pointer += 1

    def finish_trajectory(self, last_value=0):
        # Finish the trajectory by computing advantage estimates and r
        path_slice = slice(self.trajectory_start_index, self.pointer)
        rewards = np.append(self.reward_buffer[path_slice], last_value
        values = np.append(self.value_buffer[path_slice], last_value)

        deltas = rewards[:-1] + self.gamma * values[1:] - values[:-1]

        self.advantage_buffer[path_slice] = discounted_cumulative_sums
            deltas, self.gamma * self.lam
        )
        self.return_buffer[path_slice] = discounted_cumulative_sums(
            rewards, self.gamma
        )[:-1]

        self.trajectory_start_index = self.pointer

    def get(self):
        # Get all data of the buffer and normalize the advantages
        self.pointer, self.trajectory_start_index = 0, 0
        advantage_mean, advantage_std = (
            np.mean(self.advantage_buffer),
            np.std(self.advantage_buffer),
        )
        self.advantage_buffer = (self.advantage_buffer - advantage_mea
        return (
            self.observation_buffer,
            self.action_buffer,
            self.advantage_buffer,
            self.return_buffer,
```

```
            self.logprobability_buffer,
        )
```

In [4]:

```
def make_config(
        *,
        steps_per_epoch=4000,
        epochs=30,
        gamma=0.99,
        clip_ratio=0.2,
        policy_learning_rate=3e-4,
        value_function_learning_rate=1e-3,
        train_policy_iterations=80,
        train_value_iterations=80,
        lam=0.97,
        target_kl=0.01,
        hidden_sizes=(64, 64),
        render=False,
    ):
    return locals()

def build_env():
    return gym.make("CartPole-v1")


def build_buffer(obs_dim, steps_per_epoch, gamma, lam):
    return Buffer(obs_dim, steps_per_epoch, gamma=gamma, lam=lam)


def build_models(observation_dimensions, hidden_sizes, num_actions):
    observation_input = keras.Input(shape=(observation_dimensions,), d
    logits = mlp(observation_input, list(hidden_sizes) + [num_actions]
    actor = keras.Model(inputs=observation_input, outputs=logits)
    value = keras.ops.squeeze(
        mlp(observation_input, list(hidden_sizes) + [1]), axis=1
    )
    critic = keras.Model(inputs=observation_input, outputs=value)
    return actor, critic


def build_optimizers(plr, vlr):
    return (
        keras.optimizers.Adam(learning_rate=plr),
        keras.optimizers.Adam(learning_rate=vlr),
    )

def train_policy(
        actor,
        policy_optimizer,
        observations,
        actions,
        logprobabilities_old,
        advantages,
```

```
        clip_ratio,
        env,
    ):
    with tf.GradientTape() as tape:
        logits = actor(observations)
        logprobabilities_new = logprobabilities(logits, actions, env)
        ratio = keras.ops.exp(logprobabilities_new - logprobabilities_
        min_advantage = tf.where(
            advantages > 0,
            (1 + clip_ratio) * advantages,
            (1 - clip_ratio) * advantages,
        )
        policy_loss = -keras.ops.mean(
            keras.ops.minimum(ratio * advantages, min_advantage)
        )
    grads = tape.gradient(policy_loss, actor.trainable_variables)
    policy_optimizer.apply_gradients(zip(grads, actor.trainable_variab
    kl = keras.ops.mean(logprobabilities_old - logprobabilities_new)
    return kl

def train_value_function(critic, value_optimizer, observations, return
    with tf.GradientTape() as tape:
        value_loss = keras.ops.mean((returns - critic(observations)) *
    grads = tape.gradient(value_loss, critic.trainable_variables)
    value_optimizer.apply_gradients(zip(grads, critic.trainable_variab
```

In [5]:

```
def train_ppo(config):
    env = build_env()
    obs_dim = env.observation_space.shape[0]
    num_actions = env.action_space.n

    actor, critic = build_models(obs_dim, config["hidden_sizes"], num_
    policy_optimizer, value_optimizer = build_optimizers(
        config["policy_learning_rate"], config["value_function_learnin
    )
    buffer = build_buffer(obs_dim, config["steps_per_epoch"], config["

    observation, _ = env.reset()
    episode_return, episode_length = 0, 0
    metrics = []

    start = time.time()

    for epoch in range(config["epochs"]):
        sum_return, sum_length, num_episodes = 0, 0, 0

        for t in range(config["steps_per_epoch"]):
            if config["render"]:
                env.render()

            logits = actor(observation.reshape(1, -1))
            action = tf.random.categorical(logits, 1)[0, 0].numpy()
```

```
            logp_t = logprobabilities(logits, [action], env).numpy()
            value_t = critic(observation.reshape(1, -1)).numpy()

            next_obs, reward, done, _, _ = env.step(action)
            buffer.store(observation, action, reward, value_t, logp_t)

            observation = next_obs
            episode_return += reward
            episode_length += 1

            terminal = done
            if terminal or (t == config["steps_per_epoch"] - 1):
                last_value = 0 if done else critic(observation.reshape
                buffer.finish_trajectory(last_value)
                sum_return += episode_return
                sum_length += episode_length
                num_episodes += 1
                observation, _ = env.reset()
                episode_return, episode_length = 0, 0

        obs_buf, act_buf, adv_buf, ret_buf, logp_buf = buffer.get()

        # Policy update
        for _ in range(config["train_policy_iterations"]):
            kl = train_policy(
                actor,
                policy_optimizer,
                obs_buf,
                act_buf,
                logp_buf,
                adv_buf,
                config["clip_ratio"],
                env
            )
            if kl > 1.5 * config["target_kl"]:
                break

        # Value update
        for _ in range(config["train_value_iterations"]):
            train_value_function(critic, value_optimizer, obs_buf, ret

        mean_ret = float(sum_return / num_episodes)
        mean_len = float(sum_length / num_episodes)
        print(f" Epoch: {epoch+1}. Mean Return: {mean_ret}. Mean Lengt
        metrics.append(
            {"epoch": epoch + 1, "mean_return": mean_ret, "mean_length
        )

    end = time.time()

    duration = end - start
    print(f"Training completed in {duration:.2f} seconds.")
```

```
    return metrics, duration
```

The problem is considered "solved" when the average total reward for the episode reaches 195 over 100 consecutive trials.

In [13]:
```
def convergence_epoch(metrics, threshold=195):
    for m in metrics:
        if m["mean_return"] >= threshold:
            return m["epoch"]
    return None

def learning_curve_auc(metrics, max_return=200):
    returns = [m["mean_return"] for m in metrics]
    return sum(returns) / (len(returns) * max_return)

def explain(name, metrics, duration):
    conv_epoch = convergence_epoch(metrics)
    auc = learning_curve_auc(metrics)

    print(f"{name} Results:")
    print("Convergence epoch:", conv_epoch if conv_epoch else "Not rea
    print("Learning curve AUC:", round(auc, 3))
    print("Final Epoch (5) Return:", metrics[-1]["mean_return"])
    print("Final Epoch (5) Length:", metrics[-1]["mean_length"])
    print("Total Training Duration:", duration)
```

Experiment 0: Original Experiment

In [17]:
```
exp0_config = make_config(epochs=30)
exp0_metrics, exp0_duration = train_ppo(exp0_config)
```

```
/var/folders/5t/yhw_yxtj4hd3cl86y215nn0h0000gn/T/ipykernel_56775/116727
174.py:41: DeprecationWarning: Conversion of an array with ndim > 0 to
a scalar is deprecated, and will error in future. Ensure you extract a
single element from your array before performing this operation. (Depre
cated NumPy 1.25.)
  self.value_buffer[self.pointer] = value
/var/folders/5t/yhw_yxtj4hd3cl86y215nn0h0000gn/T/ipykernel_56775/116727
174.py:42: DeprecationWarning: Conversion of an array with ndim > 0 to
a scalar is deprecated, and will error in future. Ensure you extract a
single element from your array before performing this operation. (Depre
cated NumPy 1.25.)
  self.logprobability_buffer[self.pointer] = logprobability
```

```
 Epoch: 1. Mean Return: 20.61855670103093. Mean Length: 20.618556701030
93
 Epoch: 2. Mean Return: 23.668639053254438. Mean Length: 23.66863905325
4438
 Epoch: 3. Mean Return: 38.095238095238095. Mean Length: 38.09523809523
8095
 Epoch: 4. Mean Return: 58.8235294117647. Mean Length: 58.8235294117647
 Epoch: 5. Mean Return: 80.0. Mean Length: 80.0
 Epoch: 6. Mean Return: 137.93103448275863. Mean Length: 137.9310344827
5863
 Epoch: 7. Mean Return: 166.66666666666666. Mean Length: 166.6666666666
6666
 Epoch: 8. Mean Return: 181.8181818181818. Mean Length: 181.81818181818
18
 Epoch: 9. Mean Return: 222.22222222222223. Mean Length: 222.2222222222
2223
 Epoch: 10. Mean Return: 400.0. Mean Length: 400.0
 Epoch: 11. Mean Return: 571.4285714285714. Mean Length: 571.4285714285
714
 Epoch: 12. Mean Return: 444.44444444444446. Mean Length: 444.444444444
44446
 Epoch: 13. Mean Return: 1333.3333333333333. Mean Length: 1333.33333333
33333
 Epoch: 14. Mean Return: 4000.0. Mean Length: 4000.0
 Epoch: 15. Mean Return: 4000.0. Mean Length: 4000.0
 Epoch: 16. Mean Return: 4000.0. Mean Length: 4000.0
 Epoch: 17. Mean Return: 4000.0. Mean Length: 4000.0
 Epoch: 18. Mean Return: 4000.0. Mean Length: 4000.0
 Epoch: 19. Mean Return: 4000.0. Mean Length: 4000.0
 Epoch: 20. Mean Return: 4000.0. Mean Length: 4000.0
 Epoch: 21. Mean Return: 4000.0. Mean Length: 4000.0
 Epoch: 22. Mean Return: 4000.0. Mean Length: 4000.0
 Epoch: 23. Mean Return: 4000.0. Mean Length: 4000.0
 Epoch: 24. Mean Return: 4000.0. Mean Length: 4000.0
 Epoch: 25. Mean Return: 4000.0. Mean Length: 4000.0
 Epoch: 26. Mean Return: 4000.0. Mean Length: 4000.0
 Epoch: 27. Mean Return: 4000.0. Mean Length: 4000.0
 Epoch: 28. Mean Return: 4000.0. Mean Length: 4000.0
 Epoch: 29. Mean Return: 4000.0. Mean Length: 4000.0
 Epoch: 30. Mean Return: 4000.0. Mean Length: 4000.0
Training completed in 200.00 seconds.
```

In [18]:
```
explain("Original Experiment", exp0_metrics, exp0_duration)
```

```
Original Experiment Results:
Convergence epoch: 9
Learning curve AUC: 11.947
Final Epoch (5) Return: 4000.0
Final Epoch (5) Length: 4000.0
Total Training Duration: 199.9999816417694
```

Experiment 1: Reduce Training Epochs

Reduce training epochs from 30 to 5.

In [7]:
```
exp1_config = make_config(epochs=5)
exp1_metrics, exp1_duration = train_ppo(exp1_config)
```

```
/var/folders/5t/yhw_yxtj4hd3cl86y215nn0h0000gn/T/ipykernel_56775/116727
174.py:41: DeprecationWarning: Conversion of an array with ndim > 0 to
a scalar is deprecated, and will error in future. Ensure you extract a
single element from your array before performing this operation. (Depre
cated NumPy 1.25.)
  self.value_buffer[self.pointer] = value
/var/folders/5t/yhw_yxtj4hd3cl86y215nn0h0000gn/T/ipykernel_56775/116727
174.py:42: DeprecationWarning: Conversion of an array with ndim > 0 to
a scalar is deprecated, and will error in future. Ensure you extract a
single element from your array before performing this operation. (Depre
cated NumPy 1.25.)
  self.logprobability_buffer[self.pointer] = logprobability
 Epoch: 1. Mean Return: 23.668639053254438. Mean Length: 23.66863905325
4438
 Epoch: 2. Mean Return: 30.76923076923077. Mean Length: 30.769230769230
77
 Epoch: 3. Mean Return: 43.01075268817204. Mean Length: 43.010752688172
04
 Epoch: 4. Mean Return: 66.66666666666667. Mean Length: 66.666666666666
67
 Epoch: 5. Mean Return: 86.95652173913044. Mean Length: 86.956521739130
44
Training completed in 33.47 seconds.
```

In [14]:
```
explain("Experiment 1", exp1_metrics, exp1_duration)
```

```
Experiment 1 Results:
Convergence epoch: Not reached within 5 epochs
Learning curve AUC: 0.251
Final Epoch (5) Return: 86.95652173913044
Final Epoch (5) Length: 86.95652173913044
Total Training Duration: 33.47240996360779
```

> The agent has not stabilized yet at 5 epochs. Ideally `mean_reutrn` is >100.

Experiment 2: Increase Hidden Layer Size

Increase hidden layer size from (64, 64) to (128, 128)

In [9]:
```
exp2_config = make_config(epochs=5, hidden_sizes=(128, 128))
exp2_metrics, exp2_duration = train_ppo(exp2_config)
```

```
/var/folders/5t/yhw_yxtj4hd3cl86y215nn0h0000gn/T/ipykernel_56775/116727
174.py:41: DeprecationWarning: Conversion of an array with ndim > 0 to
a scalar is deprecated, and will error in future. Ensure you extract a
single element from your array before performing this operation. (Depre
cated NumPy 1.25.)
  self.value_buffer[self.pointer] = value
/var/folders/5t/yhw_yxtj4hd3cl86y215nn0h0000gn/T/ipykernel_56775/116727
174.py:42: DeprecationWarning: Conversion of an array with ndim > 0 to
a scalar is deprecated, and will error in future. Ensure you extract a
single element from your array before performing this operation. (Depre
cated NumPy 1.25.)
  self.logprobability_buffer[self.pointer] = logprobability
 Epoch: 1. Mean Return: 21.05263157894737. Mean Length: 21.052631578947
37
 Epoch: 2. Mean Return: 26.666666666666668. Mean Length: 26.66666666666
6668
 Epoch: 3. Mean Return: 35.39823008849557. Mean Length: 35.398230088495
57
 Epoch: 4. Mean Return: 62.5. Mean Length: 62.5
 Epoch: 5. Mean Return: 102.56410256410257. Mean Length: 102.5641025641
0257
Training completed in 33.57 seconds.
```

In [15]:
```
explain("Experiment 2", exp2_metrics, exp2_duration)
```

```
Experiment 2 Results:
Convergence epoch: Not reached within 5 epochs
Learning curve AUC: 0.248
Final Epoch (5) Return: 102.56410256410257
Final Epoch (5) Length: 102.56410256410257
Total Training Duration: 33.56898474693298
```

> At epoch 5, it is noticeable that the convergence speed improved as compared to experiment 1.

Experiment 3: Increase Clip Ratio

Increase clip_ratio from 0.2 to 0.4

In [11]:
```
exp3_config = make_config(epochs=5, hidden_sizes=(64, 64), clip_ratio=
exp3_metrics, exp3_duration = train_ppo(exp3_config)
```

```
/var/folders/5t/yhw_yxtj4hd3cl86y215nn0h0000gn/T/ipykernel_56775/116727
174.py:41: DeprecationWarning: Conversion of an array with ndim > 0 to
a scalar is deprecated, and will error in future. Ensure you extract a
single element from your array before performing this operation. (Depre
cated NumPy 1.25.)
  self.value_buffer[self.pointer] = value
/var/folders/5t/yhw_yxtj4hd3cl86y215nn0h0000gn/T/ipykernel_56775/116727
174.py:42: DeprecationWarning: Conversion of an array with ndim > 0 to
a scalar is deprecated, and will error in future. Ensure you extract a
single element from your array before performing this operation. (Depre
cated NumPy 1.25.)
  self.logprobability_buffer[self.pointer] = logprobability
 Epoch: 1. Mean Return: 22.857142857142858. Mean Length: 22.85714285714
2858
 Epoch: 2. Mean Return: 34.18803418803419. Mean Length: 34.188034188034
19
 Epoch: 3. Mean Return: 44.44444444444444. Mean Length: 44.444444444444
44
 Epoch: 4. Mean Return: 56.33802816901409. Mean Length: 56.338028169014
09
 Epoch: 5. Mean Return: 78.43137254901961. Mean Length: 78.431372549019
61
Training completed in 32.00 seconds.
```

In [16]:
```
explain("Experiment 3", exp3_metrics, exp3_duration)
```

```
Experiment 3 Results:
Convergence epoch: Not reached within 5 epochs
Learning curve AUC: 0.236
Final Epoch (5) Return: 78.43137254901961
Final Epoch (5) Length: 78.43137254901961
Total Training Duration: 32.00382089614868
```

> Learning is happening but it is slow as compared to previous experiments.